Pontificia Universidad Católica de Chile

Departamento de Ingeniería Mecánica y Metalúrgica



Elementos Finitos en Diseño Mecánico: Tarea 3

30 de Agosto de 2026 - Profesor Diego J. Celentano

Estudiantes: Ignacio Chacón.

Declaración de IA

Para el desarrollo de esta tarea se utilizó inteligencia artificial como herramienta de apoyo en la revisión de redacción y organización del informe.

La inteligencia artificial también se utilizó como apoyo para detectar errores de consistencia, apoyo en la creación de código y entendimiento sobre la materia.

1 Metodología

Para resolver el problema se utiliza el Método de los Elementos Finitos (MEF) mediante una formulación axisimétrica con elementos triangulares de tres nodos (T3). Se trabaja en coordenadas cilíndricas (ρ, θ, z), donde ρ corresponde a la coordenada radial y z a la dirección longitudinal del eje. Debido a la axisimetría, los desplazamientos no dependen de θ y se considera nulo el desplazamiento circunferencial. De esta forma, cada nodo posee dos grados de libertad: el desplazamiento radial u_ρ y el axial u_z.

Para un elemento triangular de nodos 1, 2 y 3, el vector de desplazamientos nodales se define como

$$\mathbf{u}^{(e)} = \begin{bmatrix} u_{\rho 1} & u_{z1} & u_{\rho 2} & u_{z2} & u_{\rho 3} & u_{z3} \end{bmatrix}^T. \tag{1}$$

Los desplazamientos dentro del elemento se aproximan mediante las funciones de forma lineales del triángulo T3. Utilizando las coordenadas naturales (ξ, η), estas funciones corresponden a

$$N_1 = 1 - \xi - \eta, \qquad N_2 = \xi, \qquad N_3 = \eta. \tag{2}$$

Por lo tanto, los campos de desplazamiento radial y axial se expresan como

$$u_\rho = \sum_{i=1}^{3} N_i u_{\rho i}, \qquad u_z = \sum_{i=1}^{3} N_i u_{zi}. \tag{3}$$

Las derivadas de las funciones de forma respecto de las coordenadas naturales se agrupan en la matriz

$$\frac{\partial \mathbf{N}}{\partial(\xi,\eta)} = \begin{bmatrix} -1 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{4}$$

Para calcular las derivadas respecto de las coordenadas físicas se utiliza la matriz jacobiana,

$$\mathbf{J} = \begin{bmatrix} \dfrac{\partial \rho}{\partial \xi} & \dfrac{\partial \rho}{\partial \eta} \\ \dfrac{\partial z}{\partial \xi} & \dfrac{\partial z}{\partial \eta} \end{bmatrix}. \tag{5}$$

Las derivadas espaciales de cada función de forma se obtienen mediante

$$\begin{bmatrix} \dfrac{\partial N_i}{\partial \rho} & \dfrac{\partial N_i}{\partial z} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial N_i}{\partial \xi} & \dfrac{\partial N_i}{\partial \eta} \end{bmatrix} \mathbf{J}^{-1}. \tag{6}$$

En la implementación, la matriz jacobiana y su determinante se calculan a partir de las coordenadas de los tres nodos de cada triángulo.

Para la formulación axisimétrica, el vector de deformaciones contiene las componentes radial, axial, circunferencial y cortante:

$$\boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_{\rho\rho} \\ \varepsilon_{zz} \\ \varepsilon_{\theta\theta} \\ \gamma_{\rho z} \end{bmatrix} = \begin{bmatrix} \dfrac{\partial u_\rho}{\partial \rho} \\ \dfrac{\partial u_z}{\partial z} \\ \dfrac{u_\rho}{\rho} \\ \dfrac{\partial u_\rho}{\partial z} + \dfrac{\partial u_z}{\partial \rho} \end{bmatrix}. \tag{7}$$

La deformación circunferencial $\varepsilon_{\theta\theta}$ debe considerarse porque un desplazamiento radial modifica la circunferencia del sólido, aun cuando no exista desplazamiento en la dirección θ.

Utilizando las funciones de forma, las deformaciones se relacionan con los desplazamientos nodales mediante

$$\boldsymbol{\varepsilon} = \mathbf{B}\mathbf{u}^{(e)}, \tag{8}$$

donde $\mathbf{B}$ es la matriz de deformación-desplazamiento:

$$\mathbf{B} = \begin{bmatrix} N_{1,\rho} & 0 & N_{2,\rho} & 0 & N_{3,\rho} & 0 \\ 0 & N_{1,z} & 0 & N_{2,z} & 0 & N_{3,z} \\ N_1/\rho & 0 & N_2/\rho & 0 & N_3/\rho & 0 \\ N_{1,z} & N_{1,\rho} & N_{2,z} & N_{2,\rho} & N_{3,z} & N_{3,\rho} \end{bmatrix}. \tag{9}$$

Aquí, $N_{i,\rho}$ y $N_{i,z}$ representan las derivadas de las funciones de forma respecto de las coordenadas físicas.

Se considera un material elástico lineal, homogéneo e isotrópico. La relación constitutiva está dada por la ley de Hooke:

$$\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}, \tag{10}$$

con el vector de tensiones

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{\rho\rho} & \sigma_{zz} & \sigma_{\theta\theta} & \tau_{\rho z} \end{bmatrix}^T. \tag{11}$$

La matriz constitutiva utilizada es

$$\mathbf{D} = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 \\ 0 & 0 & 0 & \mu \end{bmatrix}, \tag{12}$$

donde los parámetros de Lamé se calculan mediante

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \qquad \mu = \frac{E}{2(1+\nu)}. \tag{13}$$

En estas expresiones, E corresponde al módulo de Young y ν al coeficiente de Poisson.

La matriz de rigidez elemental se obtiene a partir de la formulación débil del equilibrio elástico. Para un problema axisimétrico, la integración sobre el volumen se reduce a una integral sobre la sección meridiana, incorporando el factor $2\pi\rho$ asociado a la revolución alrededor del eje:

$$\mathbf{K}^{(e)} = 2\pi \int_{A_e} \mathbf{B}^T \mathbf{D} \mathbf{B}\, \rho\, dA. \tag{14}$$

Esta integral se evalúa mediante cuadratura numérica de Gauss, utilizando tres puntos de integración sobre el triángulo de referencia:

$$\begin{aligned} (\xi_1, \eta_1) &= \left(\frac{1}{6}, \frac{1}{6}\right), \\ (\xi_2, \eta_2) &= \left(\frac{2}{3}, \frac{1}{6}\right), \qquad w_1 = w_2 = w_3 = \frac{1}{6}. \\ (\xi_3, \eta_3) &= \left(\frac{1}{6}, \frac{2}{3}\right), \end{aligned} \tag{15}$$

De esta manera, la rigidez elemental se aproxima mediante

$$\mathbf{K}^{(e)} \approx 2\pi \sum_{k=1}^{3} w_k \mathbf{B}_k^T \mathbf{D} \mathbf{B}_k\, \rho_k |\det \mathbf{J}|. \tag{16}$$

En cada punto de integración se evalúan las funciones de forma, la coordenada radial y la matriz $\mathbf{B}$. El resultado es una matriz de rigidez elemental de dimensiones 6×6, correspondiente a los seis grados de libertad del triángulo.

Las matrices elementales se ensamblan en una matriz global según los grados de libertad asociados a cada nodo. Para una malla de n_n nodos, el sistema completo contiene $2n_n$ grados de libertad y se expresa como

$$\mathbf{K}\mathbf{U} = \mathbf{F}, \tag{17}$$

donde $\mathbf{U}$ contiene los desplazamientos nodales y $\mathbf{F}$ las fuerzas externas equivalentes. La matriz global se almacena mediante una estructura dispersa para reducir el almacenamiento y facilitar la resolución del sistema.

Las cargas distribuidas sobre el contorno se incorporan mediante fuerzas nodales equivalentes. Para un segmento de borde sometido a una tracción constante $\mathbf{t}$, el vector de fuerzas se calcula como

$$\mathbf{f}_\Gamma^{(e)} = 2\pi \int_{\Gamma_e} \mathbf{N}_\Gamma^T \mathbf{t}\, \rho\, ds. \tag{18}$$

Esta integral se evalúa utilizando dos puntos de Gauss sobre el segmento de borde, distribuyendo la carga entre sus nodos.

Las condiciones de borde se incorporan restringiendo los grados de libertad correspondi-

entes. En el eje de simetría se impone $u_\rho = 0$, mientras que en el extremo de mayor diámetro se establece $u_z = 0$. Esta última condición impide el desplazamiento axial de dicho extremo, pero permite su deformación radial.

Los grados de libertad restringidos se excluyen del conjunto de incógnitas y se resuelve el sistema reducido,

$$\mathbf{K}_{LL}\mathbf{U}_L = \mathbf{F}_L, \tag{19}$$

donde el subíndice L identifica los grados de libertad libres. Posteriormente se reconstruye el vector global de desplazamientos y se calculan las reacciones mediante

$$\mathbf{R} = \mathbf{K}\mathbf{U} - \mathbf{F}. \tag{20}$$

Con los desplazamientos nodales conocidos, las deformaciones y tensiones de cada elemento se recuperan mediante

$$\boldsymbol{\varepsilon}^{(e)} = \mathbf{B}\mathbf{u}^{(e)}, \qquad \boldsymbol{\sigma}^{(e)} = \mathbf{D}\mathbf{B}\mathbf{u}^{(e)}. \tag{21}$$

Las tensiones se evalúan en los tres puntos de Gauss de cada triángulo. Aunque las funciones de forma del elemento T3 son lineales, la matriz $\mathbf{B}$ incluye el término N_i/ρ, por lo que las tensiones pueden variar dentro del elemento en la formulación axisimétrica.

Para calcular la tensión equivalente se utiliza el criterio de Von Mises, expresado a partir de las componentes del estado tensional axisimétrico:

$$\sigma_{VM} = \sqrt{\frac{(\sigma_{\rho\rho} - \sigma_{zz})^2 + (\sigma_{zz} - \sigma_{\theta\theta})^2 + (\sigma_{\theta\theta} - \sigma_{\rho\rho})^2}{2} + 3\tau_{\rho z}^2}. \tag{22}$$

Para representar gráficamente la distribución de Von Mises se utiliza el promedio de los valores obtenidos en los tres puntos de Gauss de cada elemento. En cambio, las tensiones máximas se determinan directamente a partir de los valores evaluados en esos puntos.

El factor de concentración de tensiones se calcula como

$$K_t = \frac{\sigma_{\max}}{\sigma_{\text{nom}}}, \qquad \sigma_{\text{nom}} = \frac{P}{\pi d^2/4}, \tag{23}$$

donde P es la fuerza axial y d corresponde al diámetro menor del eje. Se calculan factores de concentración tanto para la tensión equivalente de Von Mises como para la componente axial σ_{zz}.

Para aplicar esta formulación al problema se considera la sección meridiana de un eje

escalonado, caracterizado por los diámetros d y D y el radio de transición r. Las relaciones geométricas establecidas en el enunciado son

$$\frac{D}{d} = 1.25, \qquad \frac{r}{d} = 0.125. \tag{24}$$

Debido a que estas relaciones no fijan las dimensiones absolutas, se adopta $d = 1$ mm, de manera que $D = 1.25$ mm y $r = 0.125$ mm. Se consideran longitudes de $2.5d$ para ambos tramos, medidas respecto de la posición del hombro, con el propósito de separar los extremos del eje de la región donde se produce la concentración de tensiones.

La malla se construye sobre la sección meridiana, generando inicialmente puntos sobre el contorno exterior, el eje de simetría y los extremos del dominio. La transición entre diámetros se representa mediante un arco circular discretizado en segmentos rectos. Posteriormente se distribuyen puntos interiores y se genera la conectividad utilizando una triangulación de Delaunay.

Se utiliza una mayor concentración de nodos alrededor del cambio de sección, donde se esperan los mayores gradientes de tensión. En las regiones alejadas del hombro se permite una separación mayor entre puntos, evitando aumentar innecesariamente el número de elementos.

Los parámetros utilizados para generar la malla son

$$h_{\text{cerca}} = 0.01d, \qquad h_{\text{lejos}} = 0.10d, \qquad n_{\text{radial}} = 24, \qquad n_{\text{arco}} = 24. \tag{25}$$

Con estos parámetros se obtiene una malla de 1449 nodos y 2707 elementos triangulares T3, equivalente a 2898 grados de libertad antes de aplicar las restricciones. Se comprueba que los elementos tengan áreas positivas, que no existan nodos sin elementos asociados y que la triangulación respete el contorno discretizado.

La discretización busca representar con mayor detalle la geometría y la variación de tensiones en el hombro. No se realiza un estudio sistemático de convergencia, por lo que las tensiones máximas obtenidas corresponden a la malla seleccionada.

Para el material se consideran $E = 210000$ MPa y $\nu = 0.3$. La solicitación corresponde a una tensión nominal uniforme de tracción $\sigma_0 = 100$ MPa sobre el extremo de menor diámetro. La fuerza axial equivalente es

$$P = \sigma_0 \frac{\pi d^2}{4} = 78.5398 \text{ N}. \tag{26}$$

Finalmente, se restringe el desplazamiento radial sobre el eje de simetría y el desplazamiento axial en el extremo de mayor diámetro. En el extremo de menor diámetro se aplica la tracción uniforme en dirección longitudinal positiva, mientras que la superficie lateral permanece libre

de tracciones.

Pregunta 1

PROBLEMA (6 puntos)

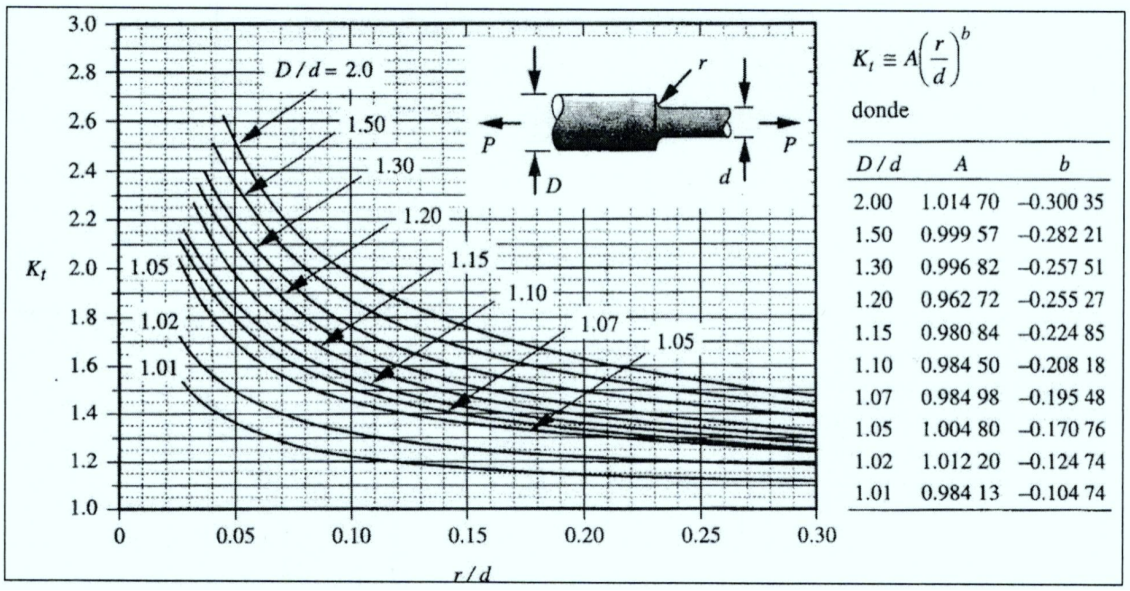
Para el eje de acero mostrado en la Figura se pide:

a) Dada una malla de elementos finitos axisimétricos de tres nodos para el caso $D/d = 1.25$ y $r/d = 0.125$, calcular bajo la condición de axisimetría la distribución del esfuerzo de Von Mises y el factor de concentración de tensiones debido a la presencia del hombro.

b) Comparar y discutir el factor de concentración obtenido en a) con el proporcionado por el gráfico adjunto.

c) Comparar y discutir el factor de concentración obtenido en a) con el proporcionado por un programa comercial de elementos finitos.

Factor de concentración de tensiones: $K_t = \sigma_{\max}/\sigma_{\text{nom}}$ con $\sigma_{\text{nom}} = \dfrac{P}{\pi d^2/4}$.



D / d	A	b
2.00	1.014 70	–0.300 35
1.50	0.999 57	–0.282 21
1.30	0.996 82	–0.257 51
1.20	0.962 72	–0.255 27
1.15	0.980 84	–0.224 85
1.10	0.984 50	–0.208 18
1.07	0.984 98	–0.195 48
1.05	1.004 80	–0.170 76
1.02	1.012 20	–0.124 74
1.01	0.984 13	–0.104 74

Figura 1. Eje con hombro sometido a tracción.

a) Distribución de tensiones y factor de concentración

La Figura 2 muestra la discretización utilizada para resolver el eje mediante elementos finitos axisimétricos T3. La malla contiene 1449 nodos y 2707 elementos triangulares, con una mayor densidad alrededor del hombro. Esta distribución permite representar con mayor detalle el filete y la variación de tensiones asociada al cambio de sección, manteniendo elementos de mayor tamaño en las regiones alejadas de la transición.

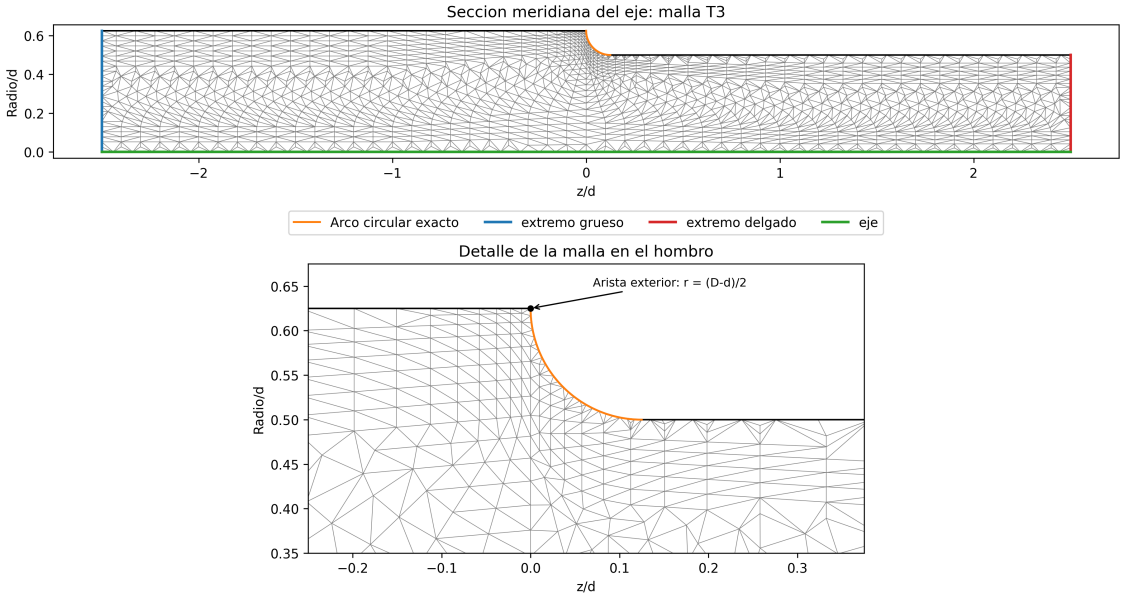


Figura 2. Malla de elementos triangulares T3 utilizada para representar la sección meridiana del eje y detalle de la transición entre diámetros.

Una vez resuelto el sistema de ecuaciones, se verificó el equilibrio global mediante la suma de las reacciones axiales en el extremo restringido. Para una fuerza aplicada de $P = 78.5398$ N se obtuvo una reacción total de $R_z = -78.5398$ N, con un error relativo de equilibrio de 1.89×10^{-13}. La coincidencia entre ambas magnitudes, con signos opuestos, confirma que el modelo satisface el equilibrio de fuerzas dentro de la precisión numérica. El desplazamiento axial máximo fue de

$$u_{z,\max} = 1.9649 \times 10^{-3} \text{ mm}, \tag{27}$$

correspondiente a un desplazamiento positivo en la dirección de la tracción aplicada.

La distribución de tensiones equivalentes de Von Mises se presenta en la Figura 3. Se observa que las mayores tensiones se desarrollan en las proximidades del hombro, mientras que

en los tramos alejados de la transición el campo resulta más uniforme. Esto se debe a que el cambio de diámetro modifica la distribución de la carga axial y genera un estado tensional localizado que no aparece en una barra cilíndrica de sección constante sometida a tracción uniforme.

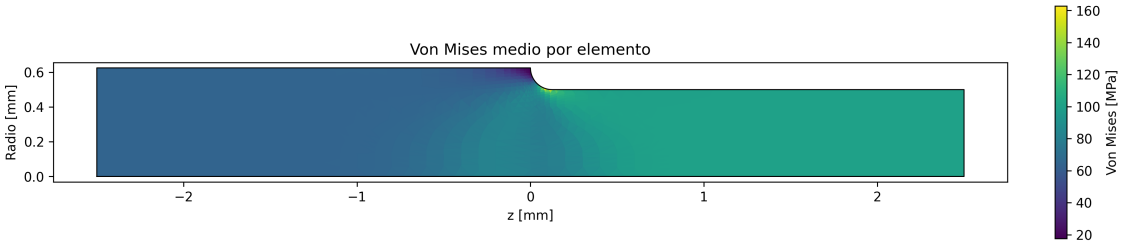


Figura 3. Distribución de la tensión equivalente de Von Mises en la sección meridiana del eje.

La Figura 4 permite observar con mayor detalle la concentración alrededor del filete. Las tensiones aumentan hacia la superficie de la transición, particularmente cerca del inicio del tramo de menor diámetro. La máxima tensión equivalente calculada fue de 162.8812 MPa y se localizó aproximadamente en $(\rho, z) = (0.4992,\ 0.1080)$ mm, en una región próxima al contorno exterior del eje.

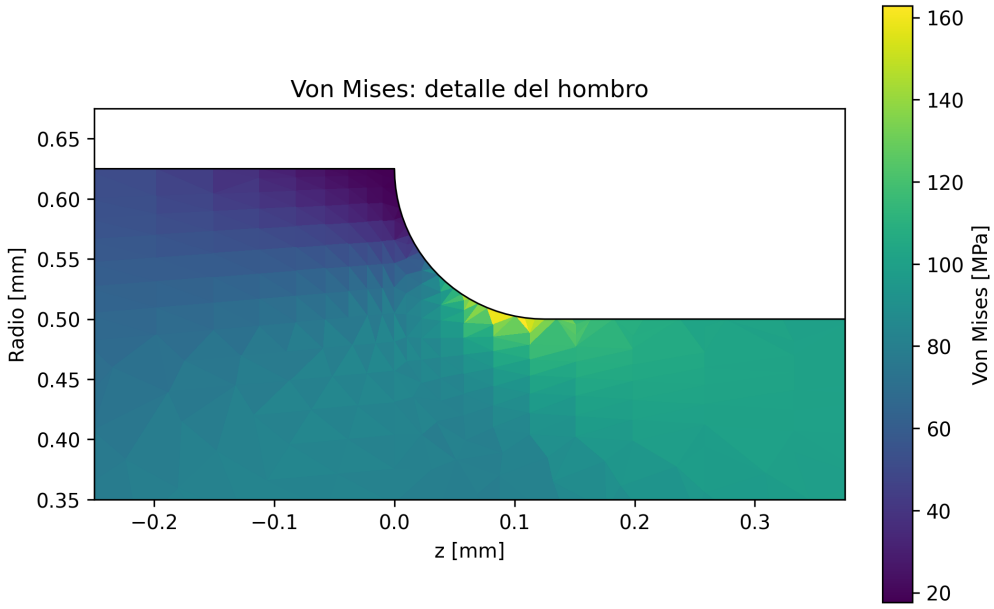


Figura 4. Detalle de la distribución de tensiones equivalentes de Von Mises en el hombro del eje.

Además de la tensión equivalente, se obtuvo la máxima componente axial, $\sigma_{zz,\max} =$

173.4778 MPa, ubicada aproximadamente en $(\rho, z) = (0.4999,\ 0.1040)$ mm. Ambos máximos se encuentran en una zona cercana, aunque sus valores difieren porque Von Mises considera conjuntamente las tensiones normales y cortantes, mientras que σ_{zz} corresponde únicamente a la tensión normal longitudinal.

Para calcular el factor de concentración se utiliza como referencia la tensión nominal en la sección de menor diámetro, definida en el enunciado como

$$\sigma_{\text{nom}} = \frac{P}{\pi d^2/4} = 100 \text{ MPa}. \tag{28}$$

A partir de las tensiones máximas obtenidas mediante MEF, los factores de concentración corresponden a

$$\boxed{K_{t,VM} = \frac{162.8812}{100} = 1.6288}, \qquad \boxed{K_{t,zz} = \frac{173.4778}{100} = 1.7348}. \tag{29}$$

Los resultados se resumen en la Tabla 1, donde se presentan las tensiones máximas y sus respectivos factores de concentración.

Tabla 1: Tensiones máximas y factores de concentración obtenidos mediante MEF.

Magnitud	Von Mises	Axial σ_{zz}
Tensión máxima [MPa]	162.8812	173.4778
Factor de concentración K_t	1.6288	1.7348

Los factores obtenidos indican que la tensión equivalente máxima supera en aproximadamente un 62.9% a la tensión nominal, mientras que la componente axial máxima presenta un incremento cercano al 73.5%. Esta diferencia refleja el efecto del hombro sobre la distribución de tensiones y muestra que el estado tensional en el filete no es estrictamente uniaxial.

Es importante considerar que estos máximos se obtuvieron evaluando las tensiones en los tres puntos de Gauss de cada elemento, mientras que los gráficos representan sus valores medios por elemento. Por lo tanto, los factores calculados dependen de la discretización y del procedimiento de evaluación utilizado, y no necesariamente coinciden con el máximo continuo sobre la superficie del filete. La comparación con el factor de concentración teórico se presenta en el inciso siguiente.

b) Comparación con el factor de concentración teórico

Para comparar los resultados obtenidos mediante MEF con la referencia proporcionada en el enunciado, se utiliza la correlación empírica del gráfico adjunto, expresada como

$$K_t \approx A\left(\frac{r}{d}\right)^b, \tag{30}$$

donde los coeficientes A y b dependen de la relación entre los diámetros D/d. Para el eje analizado se tiene $D/d = 1.25$ y $r/d = 0.125$. Como la tabla del enunciado no incluye directamente la primera relación, se realiza una interpolación lineal entre los valores correspondientes a $D/d = 1.20$ y $D/d = 1.30$, presentados en la Tabla 2.

Tabla 2: Coeficientes de la correlación empírica para las relaciones de diámetros utilizadas en la interpolación.

D/d	A	b
1.20	0.96272	-0.25527
1.30	0.99682	-0.25751

Debido a que $D/d = 1.25$ corresponde al punto medio del intervalo, los coeficientes interpolados se calculan como

$$A = \frac{0.96272 + 0.99682}{2} = 0.97977, \qquad b = \frac{-0.25527 - 0.25751}{2} = -0.25639. \tag{31}$$

Sustituyendo estos valores en la correlación, se obtiene

$$\boxed{K_{t,\text{teórico}} \approx 0.97977(0.125)^{-0.25639} = 1.6698}. \tag{32}$$

Este resultado se compara con los factores calculados mediante elementos finitos en el inciso anterior. La diferencia porcentual absoluta respecto de la correlación se determina mediante

$$\Delta K_t = \frac{|K_{t,\text{MEF}} - K_{t,\text{teórico}}|}{K_{t,\text{teórico}}} \times 100. \tag{33}$$

Los valores obtenidos se presentan en la Tabla 3, mientras que la Figura 5 muestra gráficamente las diferencias entre los tres factores.

Tabla 3: Comparación entre el factor de concentración teórico y los resultados obtenidos mediante MEF.

Método	K_t	Diferencia [%]
Correlación teórica	1.6698	–
MEF – Von Mises	1.6288	2.46
MEF – tensión axial	1.7348	3.89

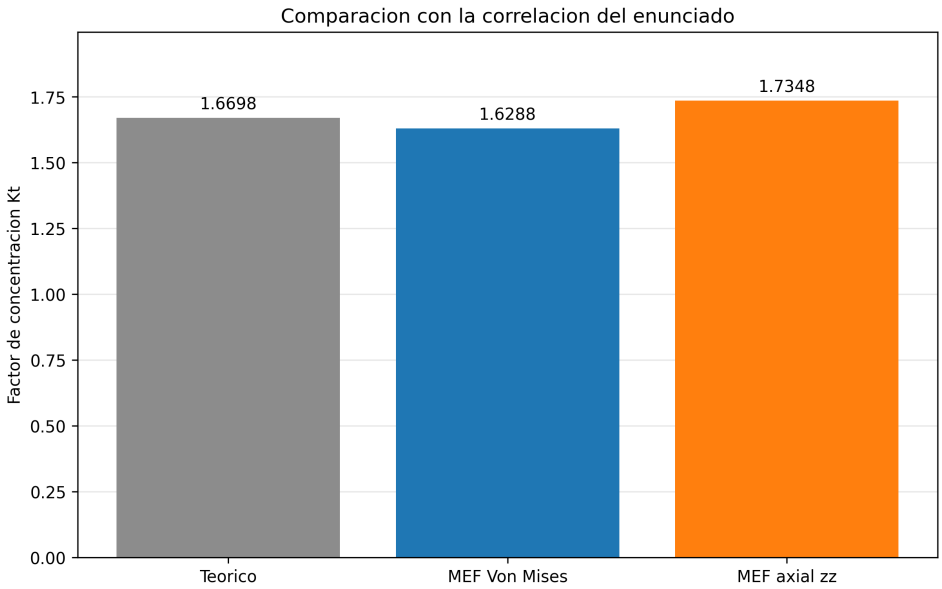


Figura 5. Comparación de los factores de concentración obtenidos mediante MEF con la correlación empírica del enunciado.

El factor obtenido a partir de Von Mises es un 2.46% inferior al valor teórico, mientras que el calculado mediante la componente axial resulta un 3.89% superior. Ambos valores son cercanos a la referencia empírica, aunque es necesario considerar que no representan exactamente la misma medida de tensión.

La correlación del enunciado corresponde a la concentración de tensión elástica producida por un hombro bajo tracción axial. Por esta razón, la comparación con la máxima componente axial σ_{zz} resulta más directa que aquella realizada con Von Mises, que combina las componentes normales y cortantes del estado tensional. Sin embargo, la dirección de la tensión normal máxima sobre el filete no necesariamente coincide con el eje longitudinal, por lo que $K_{t,zz}$ tampoco debe interpretarse como una reproducción exacta de la definición utilizada en la correlación.

Las diferencias observadas pueden relacionarse con la aproximación geométrica del filete mediante segmentos rectos, la discretización con elementos T3 y la evaluación de las tensiones en puntos de Gauss interiores. Además, la correlación representa una geometría idealizada, mientras que el modelo numérico utiliza la geometría particular definida en la metodología. Por lo tanto, la cercanía entre los resultados permite comprobar que la magnitud de la concentración calculada es razonable, pero no demuestra por sí sola la convergencia de la solución numérica.

c) Comparación con software comercial

Para contrastar los resultados obtenidos mediante el modelo de elementos finitos desarrollado en Python, se realizó un análisis estructural estático utilizando Autodesk Inventor 2026. Se construyó un modelo tridimensional del eje, manteniendo las dimensiones, propiedades del material y solicitación empleadas anteriormente. En el extremo de mayor diámetro se aplicó una restricción sin fricción, que impide el desplazamiento axial de la cara y permite su deformación transversal, mientras que en el extremo de menor diámetro se aplicó una fuerza axial de 78.5398 N.

A diferencia del modelo desarrollado en Python, que utiliza elementos triangulares T3 sobre la sección meridiana bajo la hipótesis de axisimetría, Inventor discretiza el sólido tridimensional completo mediante elementos volumétricos. La malla generada contiene 176967 nodos y 116475 elementos, cuya distribución se presenta en la Figura 6.

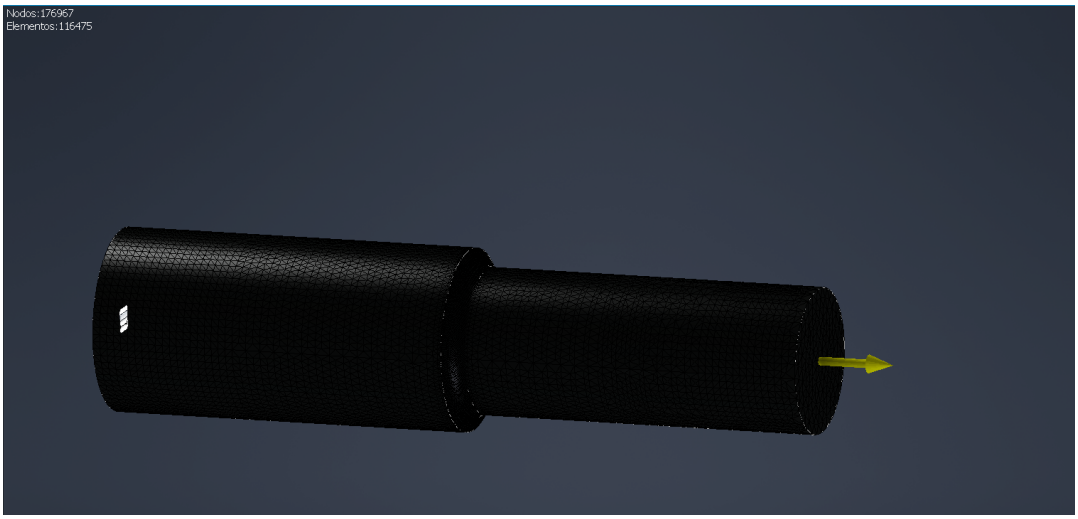


Figura 6. Malla tridimensional utilizada en el análisis estructural del eje mediante Autodesk Inventor 2026.

La distribución de tensiones equivalentes de Von Mises obtenida mediante Inventor se presenta en la Figura 7. Se observa que las tensiones aumentan en el tramo de menor diámetro y alcanzan sus valores más elevados alrededor del filete, en concordancia con el comportamiento obtenido mediante el modelo axisimétrico. La máxima tensión equivalente calculada por Inventor corresponde a

$$\sigma_{VM,\max}^{\text{Inventor}} = 168.3 \text{ MPa}. \tag{34}$$

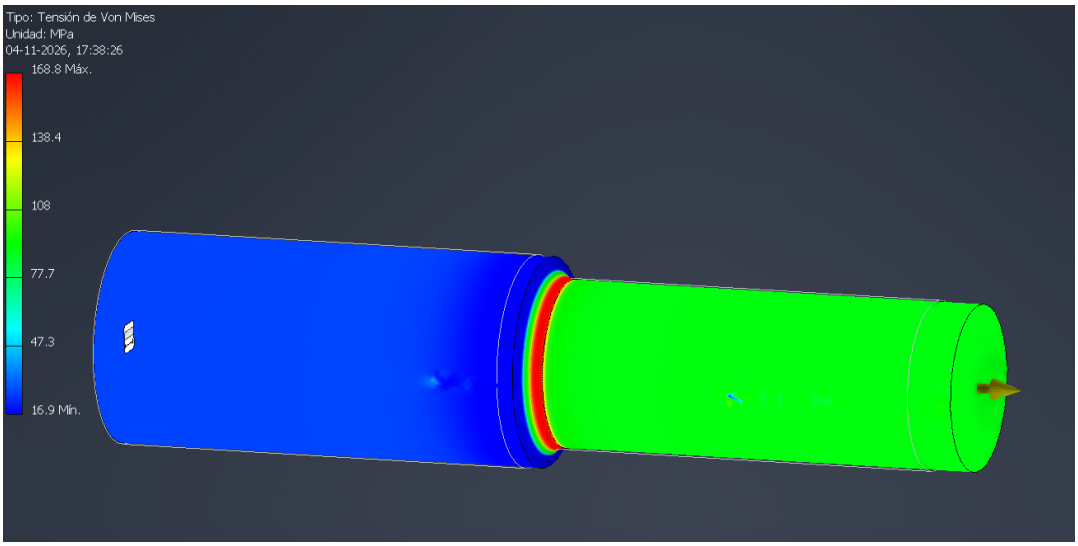


Figura 7. Distribución de tensiones equivalentes de Von Mises obtenida mediante Autodesk Inventor 2026.

De manera similar, la Figura 8 muestra la distribución de la componente normal de tensión en la dirección longitudinal del eje. Su valor máximo alcanza 177.4 MPa y se localiza también en las proximidades del cambio de sección.

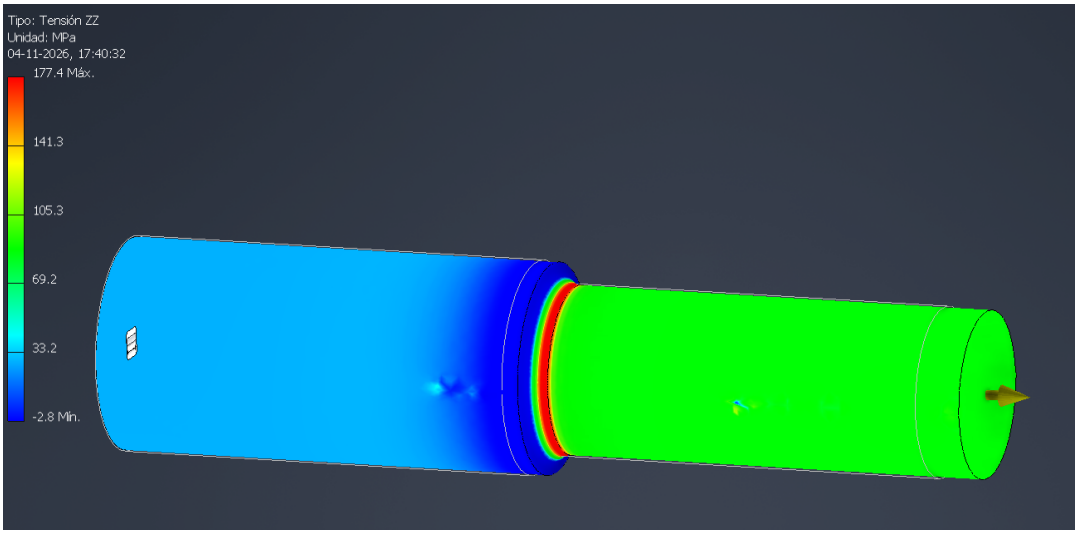


Figura 8. Distribución de la componente axial de tensión obtenida mediante Autodesk Inventor 2026.

Utilizando la misma tensión nominal de referencia, $\sigma_{\text{nom}} = 100$ MPa, los factores de concentración obtenidos mediante Inventor corresponden a

$$K_{t,VM}^{\text{Inventor}} = \frac{168.3}{100} = 1.6830, \qquad K_{t,zz}^{\text{Inventor}} = \frac{177.4}{100} = 1.7740. \tag{35}$$

La Tabla 4 presenta estos resultados junto con los calculados mediante el modelo axisimétrico T3. Las diferencias porcentuales se determinan tomando como referencia los valores entregados por Inventor.

Tabla 4: Comparación de tensiones máximas y factores de concentración obtenidos mediante Python e Inventor 2026.

Magnitud	Python T3	Inventor	Diferencia [%]
$\sigma_{VM,\max}$ [MPa]	162.88	168.30	3.22
$\sigma_{zz,\max}$ [MPa]	173.48	177.40	2.21
$K_{t,VM}$	1.6288	1.6830	3.22
$K_{t,zz}$	1.7348	1.7740	2.21

Los resultados muestran una diferencia del 3.22% en la máxima tensión equivalente de Von Mises y del 2.21% en la máxima componente axial. En ambos casos, Inventor entrega valores ligeramente superiores a los obtenidos mediante Python, aunque los dos modelos identifican la misma región crítica alrededor del hombro.

Las diferencias pueden relacionarse con las formulaciones empleadas. El modelo propio utiliza una discretización bidimensional axisimétrica con elementos triangulares lineales, mientras que Inventor resuelve la geometría tridimensional mediante una malla volumétrica. También pueden influir las diferencias en la representación del filete, las características de los elementos y los procedimientos utilizados para recuperar las tensiones máximas.

La cercanía entre los resultados resulta favorable, especialmente porque se utilizaron discretizaciones y procedimientos de resolución independientes. Sin embargo, esta comparación no constituye por sí sola una demostración de convergencia de las tensiones máximas, ya que estas pueden variar con el mallado y el método de evaluación empleado.

En conjunto, ambos modelos reproducen la concentración de tensiones producida por el cambio de sección y entregan factores de magnitud similar. Esto respalda la consistencia física de los resultados obtenidos mediante el programa desarrollado, dentro de las aproximaciones consideradas.

A Código de implementación

El código utilizado para desarrollar los incisos (a) y (b) se presenta a continuación. La implementación se realizó en Python utilizando NumPy, SciPy y Matplotlib.

```
# # Tarea 3  Elementos Finitos en Diseo Mecnico


import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.tri as mtri
from scipy.sparse import coo_matrix, csr_matrix
from scipy.sparse.linalg import spsolve


# ## a)


def funciones_forma(r,s):
    N1 = 1 - s - r
    N2 = r
    N3 = s

    dN = np.array([[-1, -1], [1, 0], [0, 1]])

    return N1, N2, N3, dN


def matriz_constitutiva(E, nu):
    D = np.array([[E*(1-nu)/((1+nu)*(1-2*nu)), E*nu/((1+nu)*(1-2*nu)), E*nu/((1+nu)*(1-2*nu)), 0],
                  [E*nu/((1+nu)*(1-2*nu)), E*(1-nu)/((1+nu)*(1-2*nu)), E*nu/((1+nu)*(1-2*nu)), 0],
                  [E*nu/((1+nu)*(1-2*nu)), E*nu/((1+nu)*(1-2*nu)), E*(1-nu)/((1+nu)*(1-2*nu)), 0],
                  [0, 0, 0, E/(2*(1+nu))]])

    return D


def matriz_jacobiana(coord, dN):
    coord = np.array(coord)
    J = coord.T @ dN
    detJ = np.linalg.det(J)
    invJ = np.linalg.inv(J)

    return J, detJ, invJ


def matriz_B(N, dN, invJ, radio):
    B = np.zeros((4, 6))
    for i in range(3):
```

```
        B[0, 2*i] = dN[i, 0] * invJ[0, 0] + dN[i, 1] * invJ[1, 0]
        B[1, 2*i+1] = dN[i, 0] * invJ[0, 1] + dN[i, 1] * invJ[1, 1]
        B[2, 2*i] = N[i]/radio
        B[3, 2*i] = dN[i, 0] * invJ[0, 1] + dN[i, 1] * invJ[1, 1]
        B[3, 2*i+1] = dN[i, 0] * invJ[0, 0] + dN[i, 1] * invJ[1, 0]
    return B


def rigidez_elemental(coord, D):
    coord = np.array(coord)
    puntos = [[1/6, 1/6], [2/3, 1/6], [1/6, 2/3]]
    pesos = [1/6, 1/6, 1/6]
    Ke = np.zeros((6, 6))

    for k in range(3):
        r, s = puntos[k]
        N1, N2, N3, dN = funciones_forma(r, s)
        N = np.array([N1, N2, N3])
        radio = N1*coord[0, 0] + N2*coord[1, 0] + N3*coord[2, 0]
        J, detJ, invJ = matriz_jacobiana(coord, dN)
        B = matriz_B(N, dN, invJ, radio)
        Ke += 2*np.pi * pesos[k] * B.T @ D @ B * radio * abs(detJ)

    return Ke


def gdl_elemento(nodos):
    gdl = []
    for i in nodos:
        gdl.append(2*i)
        gdl.append(2*i+1)

    return gdl


def ensamblar_rigidez_global(coordenadas, conectividad, E, nu):
    coordenadas = np.array(coordenadas)
    n_gdl = 2*len(coordenadas)
    filas, columnas, valores = [], [], []
    D = matriz_constitutiva(E, nu)

    for nodos in conectividad:
        Ke = rigidez_elemental(coordenadas[nodos], D)
        gdl = gdl_elemento(nodos)
        for i in range(6):
            for j in range(6):
                filas.append(gdl[i])
                columnas.append(gdl[j])
                valores.append(Ke[i, j])

```

```
    K = coo_matrix((valores, (filas, columnas)), shape=(n_gdl, n_gdl)).tocsr()
    return K


def resolver_sistema(K, F, gdl_restringidos):
    K = csr_matrix(K)
    F = np.array(F)
    n_gdl = len(F)
    gdl_libres = np.setdiff1d(np.arange(n_gdl), gdl_restringidos)

    K_libre = K[gdl_libres][:, gdl_libres]
    U = np.zeros(n_gdl)
    U[gdl_libres] = spsolve(K_libre, F[gdl_libres])
    R = K @ U - F

    return U, R


def calcular_tensiones(coords_elem, U_elem, D, puntos):
    coords_elem = np.array(coords_elem)
    sigma = np.zeros((len(puntos), 4))

    for k in range(len(puntos)):
        r, s = puntos[k]
        N1, N2, N3, dN = funciones_forma(r, s)
        N = np.array([N1, N2, N3])
        radio = N1*coords_elem[0, 0] + N2*coords_elem[1, 0] + N3*coords_elem[2, 0]
        J, detJ, invJ = matriz_jacobiana(coords_elem, dN)
        B = matriz_B(N, dN, invJ, radio)
        epsilon = B @ U_elem
        sigma[k] = D @ epsilon

    return sigma


def von_mises(sigma):
    sigma_rr = sigma[0]
    sigma_zz = sigma[1]
    sigma_tt = sigma[2]
    tau_rz = sigma[3]

    sigma_vm = np.sqrt(((sigma_rr-sigma_zz)**2 + (sigma_zz-sigma_tt)**2 + (sigma_tt-sigma_rr)**2)/2 + 3*
     tau_rz**2)

    return sigma_vm


def calidad_triangulos(coordenadas, conectividad):
    triangulos = coordenadas[conectividad]
    lados = np.zeros((len(conectividad), 3))
```

```
    for i in range(3):
        lados[:, i] = np.linalg.norm(triangulos[:, (i+1)%3]-triangulos[:, i], axis=1)

    v1 = triangulos[:, 1]-triangulos[:, 0]
    v2 = triangulos[:, 2]-triangulos[:, 0]
    areas = np.abs(v1[:, 0]*v2[:, 1]-v1[:, 1]*v2[:, 0])/2
    relaciones = np.max(lados, axis=1)**2/(2*areas)
    angulos = np.zeros((len(conectividad), 3))
    for i in range(3):
        a = lados[:, i]
        b = lados[:, (i+1)%3]
        c = lados[:, (i+2)%3]
        coseno = (b**2+c**2-a**2)/(2*b*c)
        angulos[:, i] = np.degrees(np.arccos(np.clip(coseno, -1, 1)))
    return relaciones, np.min(angulos, axis=1)


def fuerzas_borde_axisimetrico(coord, traccion):
    coord = np.array(coord)
    longitud = np.linalg.norm(coord[1]-coord[0])
    puntos = [-1/np.sqrt(3), 1/np.sqrt(3)]
    f = np.zeros(4)

    for xi in puntos:
        N = np.array([(1-xi)/2, (1+xi)/2])
        radio = N @ coord[:, 0]
        f[0::2] += 2*np.pi * N * traccion[0] * radio * longitud/2
        f[1::2] += 2*np.pi * N * traccion[1] * radio * longitud/2

    return f


def dividir_tramo(longitud, h_cerca, h_lejos):
    distancias = [0.0]
    while distancias[-1] < longitud:
        paso = min(h_lejos, h_cerca + 0.25*distancias[-1])
        siguiente = min(distancias[-1] + paso, longitud)
        if longitud - siguiente < 0.3*paso:
            siguiente = longitud
        distancias.append(siguiente)
    return np.array(distancias)


def generar_malla_eje(d, D, r_transicion, L_grueso, L_delgado,
                      h_cerca, h_lejos, n_radial, n_arco):
    z_grueso = -dividir_tramo(L_grueso, h_cerca, h_lejos)[::-1]
    angulos = np.linspace(0, np.pi/2, n_arco+1)
    z_arco = r_transicion*(1-np.cos(angulos))
    radio_arco = d/2 + r_transicion*(1-np.sin(angulos))
    z_delgado = r_transicion + dividir_tramo(L_delgado-r_transicion, h_cerca, h_lejos)
```

```
195
196     z_perfil = np.concatenate((z_grueso, z_arco[1:], z_delgado[1:]))
197     radios = np.concatenate((np.full(len(z_grueso), D/2),
198                              radio_arco[1:], np.full(len(z_delgado)-1, d/2)))
199
200     t = np.linspace(0, 1, n_radial+1)
201     fracciones = 1-(1-t)**1.5
202     coordenadas = []
203     for i in range(len(z_perfil)):
204         coordenadas.append([radios[i], z_perfil[i]])
205     nodos_exterior = np.arange(len(coordenadas))
206
207     bordes = {"nodos_exterior": nodos_exterior}
208     for nombre, radio, z, nodo_final in [
209             ("extremo_grueso", D/2, -L_grueso, nodos_exterior[0]),
210             ("extremo_delgado", d/2, L_delgado, nodos_exterior[-1])]:
211         nodos = []
212         for fraccion in t[:-1]:
213             nodos.append(len(coordenadas))
214             coordenadas.append([radio*fraccion, z])
215         nodos.append(nodo_final)
216         bordes["nodos_"+nombre] = np.array(nodos)
217
218     paso_eje = min(h_lejos, d/n_radial)
219     z_izquierda = -dividir_tramo(L_grueso, paso_eje, h_lejos)[::-1]
220     z_derecha = dividir_tramo(L_delgado, paso_eje, h_lejos)[1:]
221     z_eje = np.concatenate((z_izquierda, z_derecha))
222     nodos_eje = [bordes["nodos_extremo_grueso"][0]]
223     for z in z_eje[1:-1]:
224         nodos_eje.append(len(coordenadas))
225         coordenadas.append([0, z])
226     nodos_eje.append(bordes["nodos_extremo_delgado"][0])
227     contorno = np.array(coordenadas)
228
229
230     for i in range(1, n_radial):
231         radio_fila = D/2*fracciones[i]
232         paso = min(h_lejos, h_cerca + 0.3*max(d/2-radio_fila, 0))
233         z_izquierda = -dividir_tramo(L_grueso, paso, h_lejos)[::-1]
234         z_derecha = dividir_tramo(L_delgado, paso, h_lejos)[1:]
235         z_fila = np.concatenate((z_izquierda, z_derecha))
236         for z in z_fila[1:-1]:
237             distancia = max(-z, z-r_transicion, 0)
238             peso = max(0, 1-distancia/d)
239             posicion = (1-peso)*t[i] + peso*fracciones[i]
240             radio = np.interp(z, z_perfil, radios)*posicion
241             separacion = max(0.5*h_cerca, 0.2*min(h_lejos, h_cerca+0.25*distancia))
242             distancias = np.linalg.norm(contorno-[radio, z], axis=1)
243             if np.min(distancias) >= separacion:
244                 coordenadas.append([radio, z])
```

```
    coordenadas = np.array(coordenadas)

    conectividad = mtri.Triangulation(coordenadas[:, 0], coordenadas[:, 1]).triangles
    centros = np.mean(coordenadas[conectividad], axis=1)
    radios_limite = np.interp(centros[:, 1], z_perfil, radios)
    dentro = centros[:, 0] <= radios_limite
    conectividad = conectividad[dentro]

    bordes["nodos_eje"] = np.array(nodos_eje)
    bordes["nodos_arco"] = nodos_exterior[(z_perfil >= 0) & (z_perfil <= r_transicion)]
    for nombre in ["eje", "extremo_grueso", "extremo_delgado", "exterior", "arco"]:
        nodos = bordes["nodos_"+nombre]
        bordes[nombre] = np.column_stack((nodos[:-1], nodos[1:]))

    return coordenadas, conectividad, bordes


def comprobar_malla(coordenadas, conectividad, bordes):
    if (conectividad.ndim != 2 or conectividad.shape[1] != 3
            or not np.issubdtype(conectividad.dtype, np.integer)):
        raise ValueError("La conectividad debe contener tres indices enteros por elemento.")
    if np.any(conectividad < 0) or np.any(conectividad >= len(coordenadas)):
        raise ValueError("Hay indices fuera de la malla.")
    if not np.all(np.isfinite(coordenadas)) or len(np.unique(conectividad)) != len(coordenadas):
        raise ValueError("Hay coordenadas no finitas o nodos sin elementos.")

    triangulos = coordenadas[conectividad]
    v1 = triangulos[:, 1]-triangulos[:, 0]
    v2 = triangulos[:, 2]-triangulos[:, 0]
    areas = (v1[:, 0]*v2[:, 1]-v1[:, 1]*v2[:, 0])/2
    escala = np.max(np.ptp(coordenadas, axis=0))
    if np.any(areas <= 1e-12*escala**2):
        raise ValueError("Hay elementos degenerados o con orientacion negativa.")

    exterior = coordenadas[bordes["nodos_exterior"]]
    radios, z = exterior[:, 0], exterior[:, 1]
    tolerancia = 1e-10*escala
    if np.any(np.diff(z) <= 0):
        raise ValueError("El perfil debe avanzar en z.")
    radios_nodales = coordenadas[:, 0]
    z_nodales = coordenadas[:, 1]
    radios_limite = np.interp(z_nodales, z, radios)
    fuera_radio = (radios_nodales < -tolerancia) | (radios_nodales > radios_limite+tolerancia)
    fuera_z = (z_nodales < z[0]-tolerancia) | (z_nodales > z[-1]+tolerancia)
    if np.any(fuera_radio | fuera_z):
        raise ValueError("Hay nodos fuera del contorno discretizado.")


    aristas = np.vstack((conectividad[:, [0, 1]], conectividad[:, [1, 2]],
                         conectividad[:, [2, 0]]))
```

```
    aristas, repeticiones = np.unique(np.sort(aristas, axis=1), axis=0, return_counts=True)
    contorno = np.vstack((bordes["eje"], bordes["extremo_grueso"],
                          bordes["extremo_delgado"], bordes["exterior"]))
    contorno = np.unique(np.sort(contorno, axis=1), axis=0)
    if np.any(repeticiones > 2) or not np.array_equal(aristas[repeticiones == 1], contorno):
        raise ValueError("La triangulacion no respeta el contorno completo.")

    area_perfil = np.sum((radios[:-1]+radios[1:])*np.diff(z)/2)
    if not np.isclose(np.sum(areas), area_perfil, rtol=1e-10):
        raise ValueError("Las areas de la malla y del perfil no coinciden.")
    return areas


d = 1
D = 1.25*d
r_transicion = 0.125*d
L_grueso = 2.5*d
L_delgado = 2.5*d

h_cerca = 0.01*d
h_lejos = 0.10*d
n_radial = 24
n_arco = 24

coordenadas, conectividad, bordes = generar_malla_eje(
    d, D, r_transicion, L_grueso, L_delgado,
    h_cerca, h_lejos, n_radial, n_arco)

areas = comprobar_malla(coordenadas, conectividad, bordes)
error_arco = r_transicion*(1-np.cos(np.pi/(4*n_arco)))
relaciones, angulos_minimos = calidad_triangulos(coordenadas, conectividad)
cerca_arco = np.any(np.isin(conectividad, bordes["nodos_arco"]), axis=1)

print("Malla valida:", len(coordenadas), "nodos y", len(conectividad), "elementos T3")
print(f"Area minima: {np.min(areas):.6e}   Area total: {np.sum(areas):.6f}")
print(f"Relacion maxima: global = {np.max(relaciones):.2f}; arco = {np.max(relaciones[cerca_arco]):.2f}")
print(f"Angulo minimo: global = {np.min(angulos_minimos):.2f}; arco = {np.min(angulos_minimos[cerca_arco
    ]):.2f} grados")
print(f"Desviacion maxima de las cuerdas: {error_arco/d:.3e} d")


fig, ejes = plt.subplots(2, 1, figsize=(12, 7), constrained_layout=True)

exterior = coordenadas[bordes["nodos_exterior"]]
contorno = np.vstack(([0, -L_grueso], exterior,
                      [0, L_delgado], [0, -L_grueso]))
angulos = np.linspace(0, np.pi/2, 200)
z_arco = r_transicion*(1-np.cos(angulos))
radio_arco = d/2+r_transicion*(1-np.sin(angulos))

```

```
for ax in ejes:
    ax.triplot(coordenadas[:, 1]/d, coordenadas[:, 0]/d,
               conectividad, color="0.5", linewidth=0.45)
    ax.plot(contorno[:, 1]/d, contorno[:, 0]/d, color="black", linewidth=1.2)
    ax.plot(z_arco/d, radio_arco/d, color="tab:orange", linewidth=1.5,
            label="Arco circular exacto")
    ax.set_aspect("equal")
    ax.set_xlabel("z/d")
    ax.set_ylabel("Radio/d")

for nombre, color in [("extremo_grueso", "tab:blue"),
                      ("extremo_delgado", "tab:red"), ("eje", "tab:green")]:
    nodos = bordes["nodos_"+nombre]
    ejes[0].plot(coordenadas[nodos, 1]/d, coordenadas[nodos, 0]/d,
                 color=color, linewidth=2, label=nombre.replace("_", " "))

ejes[0].set_title("Seccion meridiana del eje: malla T3")
ejes[0].legend(loc="upper center", bbox_to_anchor=(0.5, -0.35), ncol=4)
ejes[1].set_xlim(-0.25, r_transicion/d+0.25)
ejes[1].set_ylim(d/2/d-0.15, D/2/d+0.05)
ejes[1].plot(0, D/(2*d), "ko", markersize=4)
ejes[1].annotate("Arista exterior: r = (D-d)/2", xy=(0, D/(2*d)),
                 xytext=(0.07, D/(2*d)+0.025), fontsize=9,
                 arrowprops={"arrowstyle": "->"})
ejes[1].set_title("Detalle de la malla en el hombro")
os.makedirs("Graficos", exist_ok=True)
fig.savefig("Graficos/malla_eje.png", dpi=300, bbox_inches="tight")
plt.show()


E = 210000
nu = 0.3
sigma_0 = 100
P = sigma_0*np.pi*d**2/4

n_gdl = 2*len(coordenadas)
F = np.zeros(n_gdl)
traccion = np.array([0, sigma_0])

for nodos in bordes["extremo_delgado"]:
    f = fuerzas_borde_axisimetrico(coordenadas[nodos], traccion)
    gdl = gdl_elemento(nodos)
    F[gdl] += f

fuerza_axial = np.sum(F[1::2])
assert np.isclose(fuerza_axial, P, rtol=1e-10, atol=1e-12)

gdl_restringidos = []
for nodo in bordes["nodos_eje"]:
    gdl_restringidos.append(2*nodo)
```

```
for nodo in bordes["nodos_extremo_grueso"]:
    gdl_restringidos.append(2*nodo+1)


K = ensamblar_rigidez_global(coordenadas, conectividad, E, nu)
U, R = resolver_sistema(K, F, gdl_restringidos)

reaccion_axial = np.sum(R[2*bordes["nodos_extremo_grueso"]+1])
error_equilibrio = abs(fuerza_axial+reaccion_axial)/P
uz_max = np.max(U[1::2])
assert error_equilibrio < 1e-8

print("Numero de grados de libertad:", n_gdl)
print(f"Fuerza axial aplicada: {fuerza_axial:.6f} N")
print(f"Suma de reacciones axiales: {reaccion_axial:.6f} N")
print(f"Error relativo de equilibrio: {error_equilibrio:.3e}")
print(f"Desplazamiento axial maximo: {uz_max:.6e} mm")


D_material = matriz_constitutiva(E, nu)
puntos_gauss = [[1/6, 1/6], [2/3, 1/6], [1/6, 2/3]]
n_elementos = len(conectividad)
tensiones = np.zeros((n_elementos, 3, 4))
sigma_vm = np.zeros((n_elementos, 3))
coordenadas_gauss = np.zeros((n_elementos, 3, 2))

for e in range(n_elementos):
    nodos = conectividad[e]
    coords_elem = coordenadas[nodos]
    U_elem = U[gdl_elemento(nodos)]
    tensiones[e] = calcular_tensiones(coords_elem, U_elem, D_material, puntos_gauss)

    for k in range(3):
        sigma_vm[e, k] = von_mises(tensiones[e, k])
        r, s = puntos_gauss[k]
        N1, N2, N3, dN = funciones_forma(r, s)
        coordenadas_gauss[e, k] = np.array([N1, N2, N3]) @ coords_elem

sigma_vm_media = np.mean(sigma_vm, axis=1)


sigma_nom = P/(np.pi*d**2/4)
e_vm, k_vm = np.unravel_index(np.argmax(sigma_vm), sigma_vm.shape)
e_zz, k_zz = np.unravel_index(np.argmax(tensiones[:, :, 1]), (n_elementos, 3))

sigma_vm_max = sigma_vm[e_vm, k_vm]
sigma_zz_max = tensiones[e_zz, k_zz, 1]
Kt_VM = sigma_vm_max/sigma_nom
Kt_zz = sigma_zz_max/sigma_nom
r_vm, z_vm = coordenadas_gauss[e_vm, k_vm]
```

```
r_zz, z_zz = coordenadas_gauss[e_zz, k_zz]

print(f"Tension nominal: {sigma_nom:.6f} MPa")
print(f"Maximo Von Mises: {sigma_vm_max:.6f} MPa")
print(f"Maximo esfuerzo axial: {sigma_zz_max:.6f} MPa")
print(f"Kt_VM = {Kt_VM:.6f}")
print(f"Kt_zz = {Kt_zz:.6f}")
print(f"Posicion del maximo VM: (radio, z) = ({r_vm:.6f}, {z_vm:.6f}) mm")
print(f"Posicion del maximo axial: (radio, z) = ({r_zz:.6f}, {z_zz:.6f}) mm")


triangulacion = mtri.Triangulation(coordenadas[:, 1], coordenadas[:, 0], conectividad)

for detalle, nombre, tamano in [
        (False, "von_mises_eje", (12, 3)),
        (True, "von_mises_hombro", (7, 5))]:
    fig, ax = plt.subplots(figsize=tamano, constrained_layout=True)
    mapa = ax.tripcolor(triangulacion, facecolors=sigma_vm_media, shading="flat",
                        cmap="viridis", vmin=np.min(sigma_vm_media), vmax=np.max(sigma_vm_media))
    ax.plot(contorno[:, 1], contorno[:, 0], color="black", linewidth=0.8)
    ax.set_aspect("equal")
    ax.set_xlabel("z [mm]")
    ax.set_ylabel("Radio [mm]")
    ax.set_title("Von Mises medio por elemento")
    fig.colorbar(mapa, ax=ax, label="Von Mises [MPa]", shrink=0.85)

    if detalle:
        ax.set_xlim(-0.25*d, r_transicion+0.25*d)
        ax.set_ylim(d/2-0.15*d, D/2+0.05*d)
        ax.set_title("Von Mises: detalle del hombro")

    fig.savefig("Graficos/"+nombre+".png", dpi=300, bbox_inches="tight")
    plt.show()


# ## b)


relacion_D = D/d
relacion_r = r_transicion/d
A_120, b_120 = 0.96272, -0.25527
A_130, b_130 = 0.99682, -0.25751

fraccion = (relacion_D-1.20)/(1.30-1.20)
A = A_120+fraccion*(A_130-A_120)
b = b_120+fraccion*(b_130-b_120)
Kt_teorico = A*relacion_r**b

nombres_factores = ["Teorico", "MEF Von Mises", "MEF axial zz"]
factores = np.array([Kt_teorico, Kt_VM, Kt_zz])
```

```
diferencias = 100*np.abs(factores-Kt_teorico)/Kt_teorico

print(f"Coeficientes interpolados: A = {A:.5f}, b = {b:.5f}")
print(f"{'Resultado':<18} {'Kt':>12} {'Diferencia absoluta [%]':>26}")
for nombre, factor, diferencia in zip(nombres_factores, factores, diferencias):
    print(f"{nombre:<18} {factor:12.6f} {diferencia:26.3f}")


fig, ax = plt.subplots(figsize=(8, 5), constrained_layout=True)
barras = ax.bar(nombres_factores, factores, color=["0.55", "tab:blue", "tab:orange"])
for barra, factor in zip(barras, factores):
    ax.text(barra.get_x()+barra.get_width()/2, factor+0.02,
            f"{factor:.4f}", ha="center", va="bottom")
ax.set_ylim(0, 1.15*np.max(factores))
ax.set_ylabel("Factor de concentracion Kt")
ax.set_title("Comparacion con la correlacion del enunciado")
ax.set_axisbelow(True)
ax.grid(axis="y", alpha=0.3)
fig.savefig("Graficos/comparacion_Kt_teorico.png", dpi=300, bbox_inches="tight")
plt.show()
```